

Odoo + Shopify + CRM Integration

End-to-End Practical Development Project

Build a custom Odoo module, integrate it with Shopify's Admin GraphQL API, sync products/customers/orders, connect the workflow to Odoo CRM, and finish with professional QWeb PDF reports.

Recommended stack: Odoo 19 + Python + PostgreSQL + Shopify Admin GraphQL API + QWeb

1. Project Overview

Project name: Shopify Sales & CRM Hub for Odoo

The goal is to build a real-world Odoo custom module rather than only creating a simple CRUD application. The module will act as a small integration layer between Odoo and Shopify. Shopify will provide products, customers, and sales orders; Odoo will store and manage the synchronized records, connect customers to CRM activities, and generate reports.

Shopify's current Admin API is GraphQL-based. The official documentation exposes products, customers, orders, inventory, and other resources through the Admin GraphQL API.

■cite■turn0search2■turn0search1■turn0search4■turn0search0■

For the reporting stage, Odoo's printed reports use QWeb templates and an **ir.actions.report** configuration; the official Odoo developer documentation shows the standard QWeb PDF pattern. ■cite■turn0search12■

What you will learn

- Designing an Odoo module from scratch using models, security, views, menus, actions, sequences and configuration.
- Using Odoo ORM properly: fields, relations, computed fields, constraints, create/write overrides and business methods.
- Designing a clean integration architecture instead of putting API calls directly into UI code.
- Calling an external GraphQL API from Python and handling authentication, pagination, errors and rate limits.
- Mapping Shopify IDs and data to Odoo records without creating duplicates.
- Creating customers and sales orders in Odoo from Shopify data.
- Connecting imported customers/orders to Odoo CRM opportunities and activities.
- Adding scheduled synchronization through cron jobs.
- Building logs, retry mechanisms and an integration dashboard.
- Creating QWeb PDF reports from the synchronized sales data.

2. Functional Scenario

Pretend you are developing for a company that sells products through Shopify. The company wants Odoo to become its internal sales and CRM system. A salesperson should be able to open an Odoo customer and see that the customer came from Shopify. Managers should be able to inspect imported orders, their line items and totals. The CRM team should be able to follow up with customers and prospects. Finally, management should be able to print a Shopify Sales Summary PDF from Odoo.

System	Responsibility
Shopify	Online storefront, products, customers and sales orders
Odoo custom module	Integration configuration, sync jobs, mappings and logs
Odoo Sales	Customers, products and sales orders
Odoo CRM	Leads/opportunities and follow-up activities
QWeb	Printable sales/integration reports

3. Recommended Custom Module

Module technical name: shopify_crm_bridge

Do not make the module only an API connector. Make it a small business application with its own configuration, synchronization history and reporting. This gives you substantially more Odoo development experience.

Main models to create

Model	Purpose	Important fields
shopify.instance	Shopify connection/configuration	name, shop_domain, access_token, active, last_sync_at
shopify.sync.log	Integration audit/error log	operation, status, message, model, external_id, retry_count
shopify.product.map	Shopify product/variant mapping	shopify_id, product_id, variant_id, last_synced_at
shopify.customer.map	Shopify customer mapping	shopify_id, partner_id, last_synced_at
shopify.order	Imported Shopify order header	shopify_id, order_number, partner_id, date, amount, state
shopify.order.line	Imported order lines	order_id, product_id, quantity, price_unit, line_total
crm.lead extension	Connect Shopify order/customer context to CRM	shopify_customer_id, shopify_order_count, last_shopify_order

4. Phase 1 — Environment Setup

Step 1: Prepare Odoo

- Install Odoo 19 in a development environment.
- Create a dedicated PostgreSQL database for this project.
- Create a custom addons directory and add it to the Odoo addons path.
- Enable developer mode.
- Install Sales and CRM applications.
- Create a dedicated development user with administrator rights initially.
- Verify that you can upgrade a custom module from the Apps menu or command line.

Step 2: Create the module structure

```
shopify_crm_bridge/
├── __init__.py
├── __manifest__.py
├── models/
│   ├── __init__.py
│   ├── shopify_instance.py
│   ├── shopify_product.py
│   ├── shopify_customer.py
│   ├── shopify_order.py
│   └── sync_log.py
├── services/
│   ├── __init__.py
│   ├── shopify_api.py
│   └── security/
│       ├── ir.model.access.csv
│       └── security.xml
├── data/
│   ├── ir_sequence.xml
│   └── ir_cron.xml
├── views/
│   ├── shopify_instance_views.xml
│   ├── shopify_order_views.xml
│   ├── sync_log_views.xml
│   └── menus.xml
├── wizard/
│   ├── __init__.py
│   └── sync_wizard.py
├── report/
│   ├── shopify_sales_report.xml
│   └── shopify_sales_templates.xml
└── demo/
```

5. Phase 2 — Design the Odoo Module

Step 1: Manifest

Create the module manifest with dependencies on the applications/models you actually use. Keep dependencies minimal. For this project you will normally need at least base plus the Sales and CRM modules.

```
{
    'name': 'Shopify CRM Bridge',
    'version': '19.0.1.0.0',
    'depends': ['base', 'sale_management', 'crm'],
    'data': [
        'security/security.xml',
        'security/ir.model.access.csv',
        'data/ir_sequence.xml',
        'data/ir_cron.xml',
        'views/shopify_instance_views.xml',
        'views/shopify_order_views.xml',
        'views/sync_log_views.xml',
        'views/menus.xml',
        'report/shopify_sales_report.xml',
        'report/shopify_sales_templates.xml',
    ],
    'installable': True,
    'application': True,
}
```

Step 2: Build the configuration model

Create **shopify.instance**. It should contain the store name/domain, API credentials, active flag, synchronization settings and last successful synchronization timestamps. Never hard-code a token in Python.

- Add a form view with a Connection Test button.
- Add buttons for Sync Products, Sync Customers and Sync Orders.
- Add a status field such as Connected/Not Connected.
- Add last sync timestamps for each resource.
- Store credentials securely according to your deployment/security approach.
- Add access control so only authorized users can modify integration configuration.

Step 3: Add sequences

Create a sequence for internal integration records, for example SHP/2026/00001. Use it for the Odoo-side Shopify order record or synchronization batch number. Do not use Shopify's ID as your Odoo database primary key.

Step 4: Security

- Create an Integration User group.
- Create an Integration Manager group.
- Allow normal users to read imported orders and customers if appropriate.
- Allow managers to configure instances and manually run synchronization.
- Restrict access to access tokens and configuration records.
- Test access using a non-admin user.

6. Phase 3 — Build the Shopify API Service Layer

Keep API communication in a dedicated service class such as **services/shopify_api.py**. Your Odoo models should call the service rather than constructing raw HTTP requests everywhere.

Shopify's Admin GraphQL API uses POST requests to a versioned GraphQL endpoint. The current documentation demonstrates the pattern and also notes calculated query-cost rate limiting. ■cite■turn0search5■turn0search2■

Authentication task

- Create a Shopify development store for testing.
- Create an app/custom app according to Shopify's current development flow.
- Request only the scopes needed for your project, such as product, customer and order read access.
- Generate/use a development access token as appropriate for your app setup.
- Store the token in Odoo configuration rather than in source control.
- Add a Test Connection button that performs a very small query such as shop information.
- Never commit access tokens to Git.

First API call

Start with a simple query that returns the shop name. Then build a generic request helper.

```
POST https://{shop}.myshopify.com/admin/api/{version}/graphql.json
```

```
Headers:  
Content-Type: application/json  
X-Shopify-Access-Token: <TOKEN>
```

```
Body:  
{  
  "query": "query { shop { name } }"  
}
```

The official documentation shows the versioned GraphQL endpoint and access-token header pattern.

■cite■turn0search6■

Build the service methods

```
class ShopifyApiService:  
    def __init__(self, instance):  
        self.instance = instance  
  
    def execute(self, query, variables=None):  
        # Build endpoint  
        # Add authentication headers  
        # POST JSON payload  
        # Validate HTTP response  
        # Validate GraphQL errors  
        # Return data  
        pass  
  
    def get_shop(self):  
        pass  
  
    def get_products(self, cursor=None):  
        pass  
  
    def get_customers(self, cursor=None):  
        pass  
  
    def get_orders(self, cursor=None):  
        pass
```

Error handling requirements

- Handle connection failures and timeouts.
- Handle HTTP errors.
- Handle GraphQL errors returned in the JSON response.
- Write failures to shopify.sync.log.
- Do not expose the access token in log messages.

- Use safe retries for temporary failures.
- Respect Shopify rate limits and avoid unnecessarily expensive GraphQL queries.
- Make synchronization restartable.

7. Phase 4 — Product Synchronization

Shopify products include product-level information plus variants, prices and inventory-related information. The products query supports pagination and filtering. ■cite■turn0search1■

Mapping strategy

Shopify	Odoo
Product ID	External Shopify Product ID field/mapping
Variant ID	External Shopify Variant ID field/mapping
Title	product.template.name
Variant SKU	product.product.default_code
Variant price	product.product/list price as appropriate
Vendor/type/tags	Custom fields or mapped categories
Image URL	Optional image synchronization task

Implementation tasks

- Create external ID fields or mapping models.
- Fetch products using GraphQL.
- Implement cursor-based pagination.
- For each product, search by Shopify external ID first.
- If found, update it; otherwise create it.
- Handle variants separately.
- Prevent duplicate products.
- Log created/updated/skipped counts.
- Add a manual Sync Products button.
- Test with at least 10 products and multiple variants.

8. Phase 5 — Customer Synchronization + Odoo CRM

Shopify's customer query exposes customer information and purchase history, and customer access requires the appropriate scope. Use only customer data required by your integration. ■cite■turn0search4■turn0search7■

Customer mapping

- Fetch Shopify customers with pagination.
- Map Shopify customer ID to res.partner.
- Use email as a useful secondary matching key, but do not rely on email alone.
- If an existing partner has the Shopify external ID, update it.
- If there is no mapping, attempt a controlled email match.

- If no match exists, create a partner.
- Store the Shopify customer ID on the partner or in a mapping table.
- Record the Shopify order count and last order information if useful.

CRM task

For learning purposes, do not automatically create an opportunity for every completed Shopify order. Instead, implement a business rule.

Example rule: create a CRM opportunity when a new Shopify customer places their first order above a configurable amount.

- Add configuration fields for minimum order value and whether automatic lead creation is enabled.
- When a qualifying customer is imported, search for an existing open CRM opportunity.
- If none exists, create a lead/opportunity.
- Set the partner on the opportunity.
- Add a note containing the Shopify order number and amount.
- Schedule a follow-up activity for the salesperson.
- Prevent duplicate opportunities when synchronization runs again.

9. Phase 6 — Shopify Sales Order Synchronization

This is the main integration exercise. Shopify's Order object contains customer, line item, financial and fulfillment information. The orders query supports pagination, filtering and sorting. ■cite■turn0search0■turn0search10■

Order mapping

Shopify data	Odoo target
Order ID	External Shopify Order ID
Order name/number	Shopify reference field + Odoo order reference
Created date	sale.order.date_order / custom source date
Customer	sale.order.partner_id
Line items	sale.order.line
SKU	product.product
Quantity	sale.order.line.product_uom_qty
Unit price	sale.order.line.price_unit
Financial total	sale.order amount fields / validation
Financial status	Custom Shopify status field
Fulfillment status	Custom fulfillment status field

Order import algorithm

1. Fetch a page of Shopify orders.
2. For each order:
 - a. Find existing order by Shopify external ID.
 - b. Find/create customer.
 - c. For each line:
 - find product by variant/SKU mapping
 - create or update order line
 - d. Create/update the Odoo sale.order.

- e. Store Shopify financial/fulfillment status.
- f. Create sync log.
3. Save the cursor.
4. Continue until hasNextPage = false.
5. Update last successful order sync timestamp.

For production-scale datasets, investigate Shopify bulk operations after the basic paginated implementation works. Shopify documents bulk operations for large datasets. [cite turn0search0](#)

Important duplicate-prevention test

- Run order sync once and record imported order count.
- Run it a second time.
- The second run must update/skip existing records rather than creating duplicates.
- Change one Shopify order/product/customer field.
- Run synchronization again and verify the corresponding Odoo record updates.
- Force an API error and verify the sync log records the failure.
- Fix the issue and retry without corrupting already-imported records.

10. Phase 7 — Synchronization Dashboard

Create a useful Odoo dashboard/menu instead of relying only on server logs.

- Show number of Shopify instances.
- Show last product/customer/order synchronization time.
- Show successful records in the latest run.
- Show failed records in the latest run.
- Show number of pending/retry logs.
- Provide buttons to run individual synchronizations.
- Provide a button to open synchronization logs.
- Add filters for date, status and resource type.

Suggested menu

```

Shopify Integration
  ■ ■ ■ Dashboard
  ■ ■ ■ Configuration
  ■ ■ ■ ■ ■ Shopify Instances
  ■ ■ ■ Sales
  ■ ■ ■ ■ ■ Shopify Orders
  ■ ■ ■ ■ ■ Shopify Order Lines
  ■ ■ ■ Customers
  ■ ■ ■ ■ ■ Shopify Customer Mappings
  ■ ■ ■ Products
  ■ ■ ■ ■ ■ Shopify Product Mappings
  ■ ■ ■ Logs
  ■ ■ ■ Synchronization Logs

```

11. Phase 8 — Scheduled Synchronization

Add scheduled actions using Odoo's cron mechanism.

- Product sync every 30–60 minutes.
- Customer sync every 30–60 minutes.
- Order sync every 10–30 minutes for a development project.
- Keep intervals configurable where practical.

- Do not allow overlapping synchronization jobs.
- Record start/end time, duration and result.
- Make each job safe to run repeatedly.

Incremental sync

After the initial full synchronization works, add incremental synchronization using a last-sync timestamp/filter where supported. Do not start with incremental sync. First make the full sync reliable, then optimize.

12. Phase 9 — Manual Sync Wizard

Build a transient wizard so a manager can select what to synchronize.

Wizard field	Purpose
Shopify instance	Select source store
Products	Enable product sync
Customers	Enable customer sync
Orders	Enable order sync
From date	Optional initial/incremental date
To date	Optional end date
Dry run	Validate without writing where practical

13. Phase 10 — Testing Plan

Unit-level tests

- Test external ID mapping.
- Test duplicate detection.
- Test customer matching.
- Test product matching.
- Test order line mapping.
- Test API error handling.
- Test malformed GraphQL responses.
- Test retry logic.

Integration tests

- Test connection to Shopify development store.
- Import products.
- Import customers.
- Import orders.
- Run each synchronization twice.
- Modify Shopify data and synchronize again.
- Test missing SKU/product.
- Test customer without email where possible.

- Test refunded/cancelled or unusual order statuses.
- Test pagination with enough records to produce multiple pages.

Acceptance criteria

- No duplicate Shopify products after repeated sync.
- No duplicate customers after repeated sync.
- No duplicate sales orders after repeated sync.
- Every imported order can be traced to a Shopify external ID.
- Every imported order has a customer or a documented fallback.
- Unknown products are logged clearly rather than silently ignored.
- API failures appear in synchronization logs.
- Manual synchronization works from the Odoo UI.
- Scheduled synchronization works without administrator interaction.
- CRM opportunities are created only when the configured business rule is met.

14. Phase 11 — QWeb PDF Reports

Finish the project with professional QWeb reports. Odoo's developer documentation describes printed reports as a combination of an **ir.actions.report** record and a QWeb template, typically using **web.html_container** and **web.external_layout**. ■cite■turn0search12■

Report 1 — Shopify Order PDF

Create a PDF report for a single imported Shopify order.

- Company logo and company details.
- Report title: Shopify Sales Order.
- Odoo internal order reference.
- Shopify order number.
- Shopify external order ID.
- Order date.
- Customer name, email and address.
- Order line table.
- Product/SKU.
- Quantity.
- Unit price.
- Line subtotal.
- Tax if available.
- Grand total.
- Shopify financial status.
- Shopify fulfillment status.

Report 2 — Shopify Sales Summary

Create a date-range report that summarizes imported Shopify sales.

- Date range.
- Total orders.
- Total sales amount.
- Average order value.
- Number of unique customers.
- Top products by quantity.
- Top products by sales value.
- Order status breakdown.
- A detailed order table.

Report XML structure

```
<record id="action_shopify_order_report" model="ir.actions.report">
  <field name="name">Shopify Order</field>
  <field name="model">shopify.order</field>
  <field name="report_type">qweb-pdf</field>
  <field name="report_name">shopify_crm_bridge.shopify_order_report</field>
  <field name="report_file">shopify_crm_bridge.shopify_order_report</field>
  <field name="binding_model_id" ref="model_shopify_order"/>
  <field name="binding_type">report</field>
</record>
```

QWeb template skeleton

```
<template id="shopify_order_report">
  <t t-call="web.html_container">
    <t t-foreach="docs" t-as="o">
      <t t-call="web.external_layout">
        <div class="page">
          <h2>Shopify Sales Order</h2>

          <p>
            <strong>Order:</strong>
            <span t-field="o.name"/>
          </p>

          <table class="table table-sm">
            <thead>
              <tr>
                <th>Product</th>
                <th>SKU</th>
                <th>Qty</th>
                <th>Unit Price</th>
                <th>Total</th>
              </tr>
            </thead>
            <tbody>
              <tr t-foreach="o.line_ids" t-as="line">
                <td><span t-field="line.product_id"/></td>
                <td><span t-field="line.shopify_sku"/></td>
                <td><span t-field="line.quantity"/></td>
                <td><span t-field="line.price_unit"/></td>
                <td><span t-field="line.line_total"/></td>
              </tr>
            </tbody>
          </table>
        </div>
      </t>
    </t>
  </t>
</template>
```

15. Advanced Challenge — Two-Way Integration

After one-way Shopify → Odoo synchronization is stable, implement a controlled two-way feature.

Suggested feature: Odoo updates product inventory/price and pushes a change to Shopify.

- Define exactly which Odoo fields are authoritative.
- Add a synchronization direction setting.
- Add an outbound queue instead of directly calling Shopify from every write operation.
- Process the queue through cron.
- Record request, response, timestamp and result.
- Prevent synchronization loops.
- Add a manual retry button for failed outbound operations.

This is optional but highly recommended because it teaches you the difference between simple API consumption and a reliable integration architecture.

16. Advanced Challenge — Webhooks

After polling-based synchronization works, research Shopify webhooks and implement an event-driven flow for one resource, such as orders.

- Create a webhook endpoint in Odoo.
- Validate incoming requests according to Shopify's current webhook security requirements.
- Parse the event payload.
- Create an integration event/log record.
- Process the event safely and idempotently.
- Return the appropriate HTTP response quickly.
- Move heavy processing to a background/queue strategy if needed.

17. Git & Daily Development Workflow

- Create a Git repository for the Odoo module.
- Use a meaningful branch such as main/develop and feature branches.
- Commit after each meaningful feature.
- Never commit passwords, API tokens, database dumps or private credentials.
- Write a useful commit message such as 'Add Shopify product synchronization'.
- Keep a README with installation, configuration, API setup and test instructions.
- Add screenshots of the Odoo UI to the README.
- Tag a final version when the project is complete.

18. Suggested 15-Day Execution Plan

Day	Task	Expected output
1	Environment + module skeleton	Installable Odoo module
2	Models + security	Core data models and access rules
3	Menus + views + sequences	Usable Odoo UI
4	Shopify app/API setup	Working Test Connection
5	API service layer	Reusable GraphQL client
6	Product sync	Shopify → Odoo products

Day	Task	Expected output
7	Customer sync	Shopify → Odoo partners
8	CRM automation	Lead/opportunity workflow
9	Order sync	Shopify → Odoo sales
10	Logs + error handling	Reliable integration tracking
11	Cron + wizard	Automated/manual sync
12	Testing + duplicate handling	Stable integration
13	QWeb order report	Professional order PDF
14	Sales summary report	Management PDF
15	Documentation + demo	Portfolio-ready project

19. Final Deliverables Checklist

- Custom Odoo module: shopify_crm_bridge.
- Shopify configuration screen.
- Secure API configuration.
- Test Connection feature.
- Product synchronization.
- Customer synchronization.
- Sales order synchronization.
- Shopify-to-Odoo mapping strategy.
- Odoo CRM opportunity automation.
- Synchronization logs.
- Retry/error handling.
- Scheduled synchronization.
- Manual synchronization wizard.
- Odoo access groups and security rules.
- Automated tests.
- QWeb Shopify Order PDF.
- QWeb Shopify Sales Summary PDF.
- Git repository with clean commit history.
- README with setup/configuration instructions.
- Screenshots/demo video if desired.

20. Portfolio / Interview Demonstration

At the end, demonstrate the project as a complete business flow rather than showing isolated code.

- Open Odoo and show the Shopify Integration menu.
- Configure a Shopify development store.
- Click Test Connection.

- Run Product Sync and show imported products.
- Run Customer Sync and show mapped partners.
- Run Order Sync and show imported sales orders.
- Open an imported customer and demonstrate CRM information.
- Show an automatically created opportunity from the business rule.
- Open the synchronization log and demonstrate a successful and failed operation.
- Run the same sync twice and demonstrate that duplicates are not created.
- Print the Shopify Order PDF.
- Print the Sales Summary PDF.
- Explain the API service layer, pagination, error handling and idempotency.
- Explain how you would scale the integration using bulk operations, webhooks and background jobs.

21. Important API & Architecture Notes

Use the official Shopify developer documentation as the source of truth for API versions, scopes, authentication and schema because Shopify's API evolves. The current documentation provides versioned Admin GraphQL API references and examples. [■cite■turn0search2■turn0search6■](#)

Also note that Shopify order access has specific requirements and default historical-access behavior. The current Order documentation states that only the most recent 60 days of orders are available by default unless broader access is requested. Design your initial project around recent development-store orders and investigate the appropriate access requirements if you need historical data. [■cite■turn0search0■](#)

If you later want external automation into Odoo itself, Odoo 19 introduces the JSON-2 external API. Its availability depends on the Odoo plan, so for this project keep the primary Shopify integration inside the Odoo custom module unless you specifically need an external service. [■cite■turn0search13■](#)

22. Definition of Done

Consider the project complete only when all of the following are true:

- A fresh Odoo database can install the module without errors.
- A Shopify development store can be configured without editing Python source code.
- The connection test succeeds.
- Products synchronize without duplicates.
- Customers synchronize without duplicates.
- Orders synchronize without duplicates.
- CRM automation works according to the configured business rule.
- A failed API request is visible in the sync log.
- A repeated synchronization is safe.
- Cron synchronization works.
- The order PDF is professionally formatted.
- The sales summary PDF is professionally formatted.
- The README explains installation and configuration.
- The Git repository contains no secrets.
- You can explain the architecture and every major design decision in an interview.

23. Official Documentation to Keep Open While Building

Odoo developer documentation: ■url■Building a Module — Odoo 19■
19■<https://www.odoo.com/documentation/19.0/developer/tutorials/backend.html>■

Shopify Admin GraphQL API: ■url■Shopify GraphQL Admin API reference■
reference■<https://shopify.dev/docs/api/admin-graphql/latest>■

Shopify Products API: ■url■Shopify Products query documentation■
documentation■<https://shopify.dev/docs/api/admin-graphql/latest/queries/products>■

Shopify Customers API: ■url■Shopify Customers query documentation■
documentation■<https://shopify.dev/docs/api/admin-graphql/latest/queries/customers>■

Shopify Orders API: ■url■Shopify Orders documentation■
documentation■<https://shopify.dev/docs/api/admin-graphql/latest/objects/Order>■

24. Final Advice

Do this project in stages. Do not try to build the complete integration on day one. The strongest learning path is: **Odoo module** → **clean models/views/security** → **API service** → **products** → **customers** → **CRM** → **orders** → **logs/retries** → **cron** → **QWeb reports** → **advanced webhooks/two-way sync**. If you complete every stage and can explain why you designed it that way, you will have a strong practical Odoo integration project suitable for a portfolio and technical interview.